conditions aren't so interesting and aren't a cause of bugs. The C++ Standard also defines the term *data race* to mean the specific type of race condition that arises because of concurrent modification to a single object (see section 5.1.2 for details); data races cause the dreaded *undefined behavior*.

Problematic race conditions typically occur where completing an operation requires modification of two or more distinct pieces of data, such as the two link pointers in the example. Because the operation must access two separate pieces of data, these must be modified in separate instructions, and another thread could potentially access the data structure when only one of them has been completed. Race conditions can often be hard to find and hard to duplicate because the window of opportunity is small. If the modifications are done as consecutive CPU instructions, the chance of the problem exhibiting on any one run-through is very small, even if the data structure is being accessed by another thread concurrently. As the load on the system increases, and the number of times the operation is performed increases, the chance of the problematic execution sequence occurring also increases. It's almost inevitable that such problems will show up at the most inconvenient time. Because race conditions are generally timing sensitive, they can often disappear entirely when the application is run under the debugger, because the debugger affects the timing of the program, even if only slightly.

If you're writing multithreaded programs, race conditions can easily be the bane of your life; a great deal of the complexity in writing software that uses concurrency comes from avoiding problematic race conditions.

3.1.2. Avoiding problematic race conditions

There are several ways to deal with problematic race conditions. The simplest option is to wrap your data structure with a protection mechanism, to ensure that only the thread actually performing a modification can see the intermediate states where the invariants are broken. From the point of view of other threads accessing that data structure, such modifications either haven't started or have completed. The C++ Standard Library provides several such mechanisms, which are described in this chapter.